

Learn few algo designing techniques.

- divide & Conquer.
- Dynamic Programming
- Greedy technique.

⊗ After designing an algo we must do the following

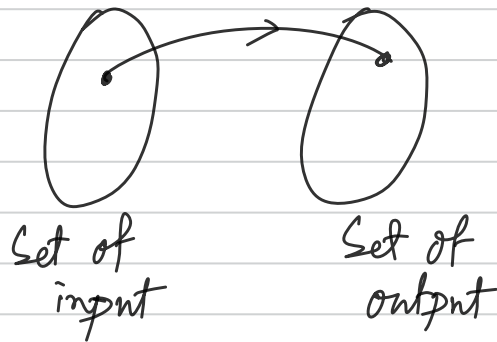
- Correctness of algo. (Mathematically)
- Efficient algo. (running time)

⊗ Data Structures: Binary Search trees, Heaps, AVL trees, Hashing..

⊗ Graph algo: BFS-DFS, Spanning trees, Shortest path,

⊗ NP-Completeness: Deterministic ✗ Non-det. algo.
NP, NP-hard;
Polynomially reducible.

Computational Probs:



Sorting Problem:

Input: A set of objects $a_1, a_2, \dots, a_n$ from an well-ordered set.

Output: Those objects arranged in inc/dec order.

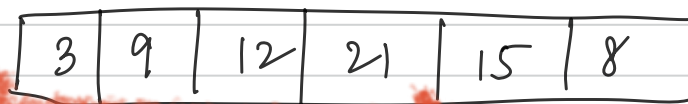
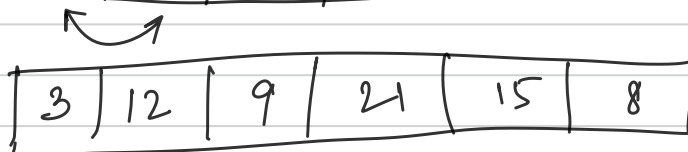


Input: 7, 13, 2, 6, 1, 21 $\in \mathbb{Z}$

Output: 1, 2, 6, 7, 13, 21

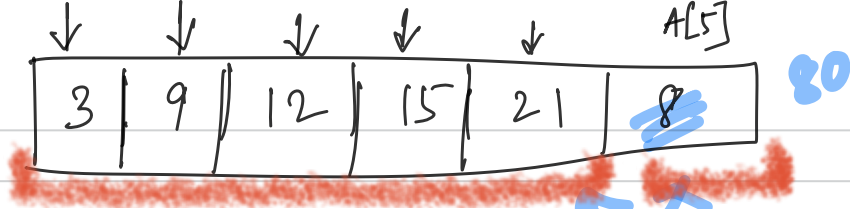
Algorithm: It's a set of well-defined rules for solving some computational problem.

Insertion Sort



sorted

unsorted

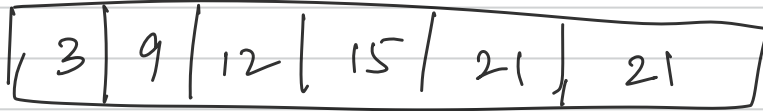


temp = 8

$(21, 8)$

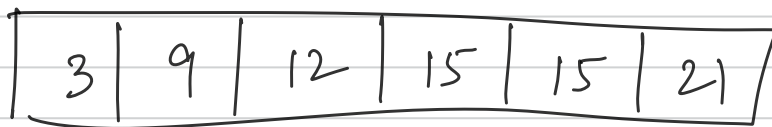
$(A[4], temp)$

$A[5] = 21$
 $A[5] = A[4]$



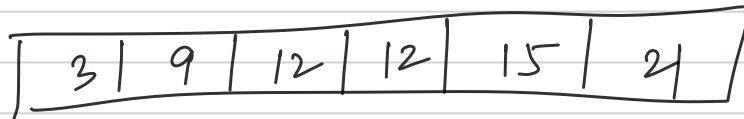
$(A[3], temp)$
 $15, 8$

$A[4] = A[3]$



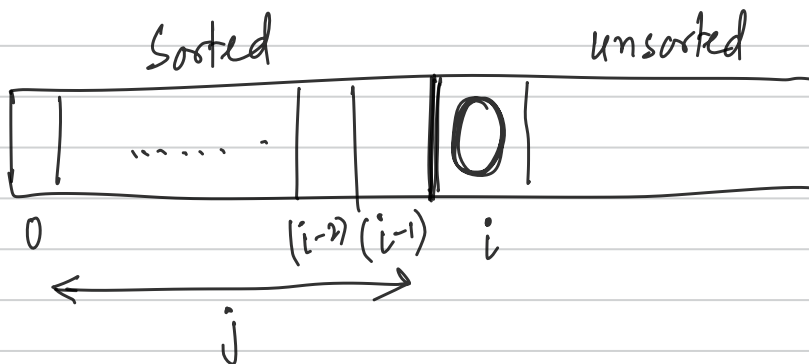
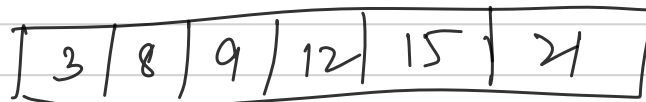
$(A[2], temp)$
 $12, 8$

$A[3] = A[2]$



Similarly, $[3, 9, 9, 12, 15, 21]$

$(3, 8)$
 $(A[0], temp)$



$temp = A[i]$

Insertion Sort:

```
1. for i = 1 to (n-1) do n
2. { temp = A[i] (n-1)
3.   j = i-1 (n-1)
4.   while (j > 0) and (A[j] > temp) do  $\sum_{i=1}^{n-1} t_i$ 
5.   { A[j+1] = A[j]  $\sum_{i=1}^{n-1} (t_i - 1)$ 
6.     j = j-1  $\sum_{i=1}^{n-1} (t_i - 1)$ 
7.   A[j+1] = temp. (n-1)
```

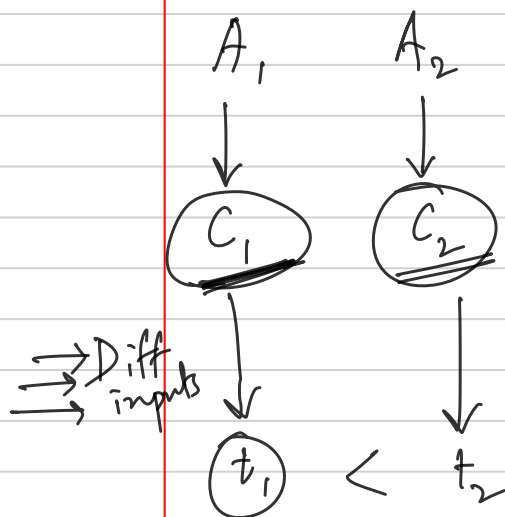
Correctness follows from mathematical induction.

Efficiency:

Write a code

Check on different input

Measure the system clock.



Drawbacks ① Need to implement.

② System dependent.

③ infinite input

⊗ We calculate the number of primitive operations performed in terms of input size.

Input: size is the total no of bits needed to represent the input.

(For sorting prob, we take the input size to be n)

$$\log_2 1000$$

⊗ t_i = no of times while loop condition is checked.

Ex. Calculate the total no of primitive operation done in Insertion Sort.